

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для интерактивной работы с
учебным расписанием на основе искусственного интеллекта»

Выполнил: Деев Егор Викторович

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Дееву Егору Викторовичу,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для интерактивной работы с учебным расписанием на основе искусственного интеллекта»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих систем управления учебным расписанием	8	05.04.2026	
Задача 1.2. Анализ инструментов разработки (Flask, PostgreSQL, OpenAI API)	6	12.04.2026	
Задача 1.3. Формулировка цели, задач и требований к приложению	6	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории и пользовательских требований	5	25.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	02.05.2026	
Задача 2.3. Проектирование модели данных: сущности User, Schedule, Metric, Share (ER-диаграмма, схемы БД)	8	09.05.2026	Не менее 3 сущностей в приложении
Задача 2.4. Разработка макетов страниц (Wireframe) интерфейса расписания и трекеров	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры проекта и вёрстка шаблонов страниц	6	21.05.2026	
Задача 3.2. Реализация аутентификации, авторизации и системы ролей пользователей (владелец / редактор / наблюдатель)	8	24.05.2026	Аутентификация и авторизация пользователей; система проверки прав доступа по роли
Задача 3.3. Реализация CRUD-интерфейса для расписаний, предметов и метрик успеваемости	8	27.05.2026	CRUD-интерфейс
Задача 3.4. Разработка сервиса загрузки изображения расписания и его распознавания с помощью ИИ	8	31.05.2026	Загрузка файлов
Задача 3.5. Реализация интерактивных трекеров успеваемости и агрегирования данных по предметам	10	03.06.2026	Агрегирование данных
Задача 3.6. Разработка системы совместного доступа и экспорта расписания	5	06.06.2026	Выгрузка файлов
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	03.04.2026	
Задача 4.2. Деплой приложения на хостинг	4	08.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания «____» _____ 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«____» _____ 2026 г. _____ Е. В. Деев
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Обзор существующих систем управления учебным расписанием	8
1.2 Анализ инструментов разработки	9
1.3 Формулировка цели, задач и требований к приложению	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	13
2.1 Анализ целевой аудитории и пользовательских требований . .	13
2.2 Описание функциональности приложения	14
2.3 Проектирование модели данных	16
2.4 Разработка макетов страниц	20
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	23
3.1 Разработка базовой структуры проекта и вёрстка шаблонов . .	23
3.2 Реализация аутентификации и системы ролей	23
3.3 Реализация CRUD-интерфейса расписаний и предметов	24
3.4 Сервис распознавания расписания с помощью AI	25
3.5 Система трекеров успеваемости	27
3.6 Система совместного доступа и экспорта расписания	27
3.7 Публикация приложения	30
ЗАКЛЮЧЕНИЕ	31
СПИСОК ЛИТЕРАТУРЫ	33

ВВЕДЕНИЕ

Актуальность темы. В условиях возрастающей интенсивности учебного процесса студентам всё сложнее самостоятельно отслеживать расписание занятий, контролировать успеваемость и планировать подготовку к экзаменам. Расписание, размещаемое вузом на официальном портале, доступно исключительно в режиме чтения и не поддаётся персонализации. Традиционные инструменты — универсальные календари и планировщики — не учитывают специфику учебного процесса: типизацию занятий (лекция, практика, лабораторная работа), ролевую модель совместного доступа и персональные трекеры успеваемости. Разработка специализированного веб-приложения, интегрирующего автоматическое распознавание расписания из изображения с помощью искусственного интеллекта и гибкую систему трекинга учебных показателей, является актуальной практической задачей.

Степень разработанности проблемы. На рынке существует ряд инструментов для работы с учебным расписанием: специализированные мобильные приложения (My Study Life, iStudiez Pro), системы управления обучением (Moodle, Canvas) и универсальные планировщики (Google Calendar, Notion). Ни одно из рассмотренных решений не объединяет в рамках единого инструмента автоматическое распознавание расписания из изображения, персональные настраиваемые трекеры успеваемости и гибкую систему совместного доступа с ролевой моделью.

Объект исследования — процессы планирования и отслеживания учебной деятельности студентов.

Предмет исследования — программные средства для автоматизации работы с учебным расписанием и трекинга успеваемости на основе веб-технологий и искусственного интеллекта.

Целью данной работы является разработка веб-приложения для интерактивной работы с учебным расписанием, обеспечивающего автоматическое извлечение данных из изображения с помощью ИИ, ведение персональных трекеров успеваемости и гибкое совместное использование расписания.

Для достижения поставленной цели необходимо решить следующие задачи: реализовать систему аутентификации и ролевого доступа; разработать сервис распознавания расписания из изображения на основе AI Vision API; создать CRUD-интерфейс для управления расписаниями, предметами и мет-

риками; реализовать систему из 11 типов трекеров успеваемости; обеспечить совместный доступ к расписанию; реализовать экспорт расписания в форматы JSON, CSV, PDF и PNG.

Практическая значимость работы состоит в том, что разработанное приложение может быть использовано студентами любого вуза для автоматизации ведения расписания и учёта академических показателей без ручного ввода данных.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих систем управления учебным расписанием

Среди инструментов, применяемых студентами для работы с расписанием и учебными задачами, можно выделить пять основных категорий.

Универсальные планировщики и календари. Google Calendar и Microsoft Outlook Calendar применяются студентами для фиксации занятий, однако ориентированы на корпоративный сценарий: ввод данных исключительно ручной, типизация занятий (лекция, практика, лабораторная работа) отсутствует, а ролевая модель «владелец — редактор — наблюдатель» не поддерживается.

Системы управления обучением (LMS). Moodle [1], Blackboard и Canvas управляются на стороне учебного заведения: студент получает расписание в режиме «только чтение» и является пассивным потребителем данных. Настраиваемые персональные трекеры успеваемости и загрузка расписания из изображения в них не предусмотрены.

Институциональные системы российских вузов. Порталы расписания на базе 1С:Университет предоставляют актуальное расписание группы в режиме «только чтение» без возможности персонализации, ведения заметок или создания трекеров прогресса.

Мобильные приложения для студентов. My Study Life [2], Class Timetable и iStudiez Pro специализированы под студенческий сценарий, однако требуют полностью ручного ввода данных, а набор встроенных типов трекеров фиксирован и не расширяем.

Инструменты личной продуктивности. Notion и TickTick позволяют строить произвольные базы данных, что технически покрывает учебные сценарии, но требует значительных усилий по настройке. Специализированной структуры расписания и AI-интеграции для обработки изображений в них нет.

В таблице 1.1 приведено сравнение рассмотренных систем по ключевым критериям.

Таблица 1.1 – Сравнительный анализ существующих систем

Система	AI-распознавание	Личные трекеры	Ролевой доступ	Учебная специфика	Бесплатно
Google Calendar	—	—	частично	—	да
Moodle (LMS)	—	—	да	да	да
Портал расписания вуза	—	—	—	да	да
My Study Life	—	частично	—	да	частично
Notion	—	частично	да	—	частично
Разрабатываемое приложение	да	да	да	да	да

Ни одно из рассмотренных решений не объединяет в рамках единого инструмента автоматическое распознавание расписания из изображения, персональные настраиваемые трекеры успеваемости и гибкую систему совместного доступа с ролевой моделью, что определяет актуальность разрабатываемого веб-приложения.

1.2 Анализ инструментов разработки

Выбор технологического стека определялся требованиями к гибкости архитектуры, поддержке JSON-данных переменной структуры, возможности интеграции с AI API и доступности инструментов в российской инфраструктуре.

Веб-фреймворк. Для реализации серверной части рассматривались три Python-фреймворка: Flask, Django и FastAPI. Django навязывает жёсткую структуру проекта и включает избыточный функционал. FastAPI обеспечивает нативный async, однако не имеет встроенного шаблонизатора для серверного рендеринга HTML. Сравнение представлено в таблице 1.2.

Flask [3] выбран как оптимальный вариант: встроенный Jinja2 обеспечивает серверный рендеринг с поддержкой макросов, а расширения Flask-Login, Flask-WTF и Flask-SQLAlchemy покрывают все потребности проекта без избыточных зависимостей.

Система управления базами данных. При выборе СУБД рассматривались PostgreSQL, MySQL и MongoDB. MySQL не поддерживает JSONB-

Таблица 1.2 – Сравнение веб-фреймворков

Критерий	Flask	Django	FastAPI
Встроенный шаблонизатор	Jinja2	Jinja2	нет
Встроенная ORM	нет	да	нет
Управление сессиями	Flask-Login	встроенное	нет (JWT)
Поддержка async	нет	частично	нативная
Гибкость структуры	высокая	низкая	высокая

индексирование, необходимое для хранения данных расписания переменной структуры. MongoDB обеспечивает гибкое хранение документов, однако не поддерживает полноценные ACID-транзакции и не интегрируется с SQLAlchemy. Сравнение представлено в таблице 1.3.

Таблица 1.3 – Сравнение систем управления базами данных

Критерий	PostgreSQL	MySQL	MongoDB
Модель данных	реляционная	реляционная	документоориент.
Тип JSONB / GIN-индекс	да	нет	да (BSON)
ACID-транзакции	да	да	частично
Поддержка SQLAlchemy	полная	полная	нет
Сложные типы данных	высокая	ограниченная	высокая

PostgreSQL [4] выбран благодаря сочетанию реляционной модели для таблиц пользователей, метрик и прав доступа с типом JSONB для хранения гибкой вложенной структуры данных расписания, а также полной поддержке SQLAlchemy [5] и ACID-гарантий целостности данных.

AI API для обработки изображений. Для задачи распознавания расписания из изображения рассматривались три мультимодальные языковые модели. GPT-4o демонстрирует высокую точность, однако прямой доступ к API из России затруднён. Llama 3.2 Vision доступна для локального развёртывания, но уступает в точности при анализе табличных данных. Сравнение представлено в таблице 1.4.

Таблица 1.4 – Сравнение AI Vision API

Критерий	Gemini 3.1 Flash	GPT-4o	Llama 3.2 Vision
Мультимодальность	да	да	да
Совм. с OpenAI API	да	да	частично
Доступ из России	да	нет	локально
Стоимость запросов	низкая	высокая	бесплатно
Точность (таблицы)	высокая	высокая	средняя

Модель Gemini 3.1 Flash [6] выбрана как оптимальная: она доступна через российского провайдера Timeweb Cloud AI по OpenAI-совместимому интерфейсу, что обеспечивает переносимость кода при смене провайдера, и сочетает высокую точность анализа изображений с низкой стоимостью запросов.

Клиентская часть. В качестве альтернатив рассматривались фреймворки React и Vue.js, однако их применение избыточно для данного приложения, в котором большинство страниц формируется на сервере. Интерфейс реализован на Bootstrap 5.3 с использованием Vanilla JS и Fetch API: это позволяет обновлять виджеты метрик без перезагрузки страницы при минимальном объёме клиентского кода.

1.3 Формулировка цели, задач и требований к приложению

Цель работы — разработать веб-приложение для интерактивной работы с учебным расписанием, обеспечивающее автоматическое извлечение данных из изображения с помощью ИИ, ведение персональных трекеров успеваемости и гибкое совместное использование расписания.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Реализовать систему аутентификации и авторизации пользователей.
2. Разработать ролевую модель доступа: владелец — полный контроль; редактор — редактирование описаний и трекеров; наблюдатель — только просмотр.
3. Реализовать CRUD-интерфейс для расписаний, предметов и метрик.

4. Разработать сервис загрузки изображения расписания и его автоматического распознавания с помощью AI Vision API.
5. Реализовать систему трекеров успеваемости с поддержкой 11 типов метрик и агрегированием данных по предмету.
6. Разработать систему совместного доступа с тремя режимами: просмотр, редактирование и использование расписания как шаблона.
7. Реализовать объединение нескольких расписаний с разрешением конфликтов предметов.
8. Реализовать экспорт расписания в форматы JSON, CSV, PDF и PNG.
9. Обеспечить деплой приложения на хостинг.

Сформулированные цель и задачи определяют границы разрабатываемой системы и служат основой для последующего этапа — проектирования веб-приложения.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории и пользовательских требований

Разрабатываемое приложение не ограничено университетской аудиторией: его ключевая функциональность — структурированное расписание с трекерами прогресса и совместным доступом — востребована везде, где нужно планировать занятия и отслеживать результаты. Выделяются три категории пользователей.

Студенты — основная категория. Загружают расписание из фотографии, отслеживают посещаемость и оценки с помощью трекеров.

Люди, самостоятельно изучающие новые навыки — слушатели онлайн-курсов, участники кружков и секций. Составляют расписание вручную, добавляют трекеры прогресса и отслеживают серии непрерывных занятий.

Репетиторы и тренеры — ведут расписание с несколькими учениками. Передают расписание по ссылке в режиме «только чтение», фиксируют пройденные темы с помощью трекеров.

Исходя из потребностей перечисленных категорий, выделяются три функциональные роли в системе:

Владелец расписания — пользователь, создавший расписание. Обладает полным контролем: добавляет и редактирует предметы, слоты и трекеры, управляет совместным доступом, удаляет расписание.

Редактор — пользователь, получивший ссылку с правами на редактирование. Может редактировать описания предметов и трекеры успеваемости, но не изменяет структуру расписания и не управляет общим доступом.

Наблюдатель — пользователь, перешедший по ссылке с правами на просмотр. Видит расписание и состояние трекеров в режиме «только чтение».

Ключевые пользовательские истории (User Story):

- «Я, как студент, хочу сфотографировать расписание и получить его в структурированном виде, не вводя данные вручную»;
- «Я, как студент, хочу добавить трекер посещаемости к предмету, чтобы отслеживать, сколько занятий я посетил»;
- «Я, как слушатель онлайн-курса, хочу составить расписание занятий и отслеживать серию непрерывных дней активности»;

- «Я, как репетитор, хочу передать ученику расписание по ссылке, чтобы он видел план занятий без возможности его изменить».

2.2 Описание функциональности приложения

Функциональные возможности системы охватывают семь основных блоков, сформированных на основе требований трёх категорий пользователей.

Аутентификация и авторизация. Регистрация и вход по логину и паролю. Пароли хранятся в хешированном виде (Werkzeug [7]). Сессии управляются библиотекой Flask-Login [8]. Формы защищены от CSRF-атак с помощью Flask-WTF [9].

Загрузка и распознавание расписания. Пользователь загружает фотографию или скан расписания (JPG, PNG, WebP, BMP, до 20 МБ). Сервис передаёт изображение в base64-кодировке в мультимодальную языковую модель через AI Vision API [10]. Модель возвращает структурированный JSON, который сохраняется в базе данных.

Управление расписанием (CRUD). Полный набор операций над структурой расписания: переименование предметов и периодов дат, добавление, изменение и удаление слотов занятий, смена типа занятия, удаление предметов. Данные структуры расписания хранятся в поле типа JSON.

Трекеры успеваемости. Для каждого предмета создаётся набор виджетов-метрик. Поддерживаются 11 типов трекеров (контрольные точки, посещаемость, оценки и др.). Виджеты размещаются на дашборде с сеткой 12 колонок; положение и размер каждого виджета сохраняются в базе данных.

Объединение расписаний. Функция слияния двух и более расписаний в одно с разрешением конфликтов по предметам. Позволяет объединить расписания нескольких курсов или подгрупп в единое сводное расписание.

Совместный доступ. Владелец формирует токен-ссылку с одним из трёх типов доступа: просмотр (view), редактирование (edit) или клонирование как шаблон (template). Режим шаблона позволяет получателю скопировать расписание к себе и адаптировать его под свои нужды.

Экспорт расписания. Пользователь может выгрузить расписание в одном из четырёх форматов непосредственно с карточки на странице профиля. Формат JSON содержит исходную структуру данных расписания. Формат CSV представляет расписание в виде плоской таблицы, пригодной для открытия в табличных редакторах. Форматы PDF и PNG формируются путём серверного

рендеринга HTML-шаблона с помощью headless-браузера Playwright [11]: PDF — постраничный документ в альбомной ориентации формата A4; PNG — изображение с недельной сеткой занятий, визуально повторяющей интерфейс страницы расписания, с фильтрацией по актуальным периодам дат.

На рисунке 2.1 представлена диаграмма вариантов использования системы.

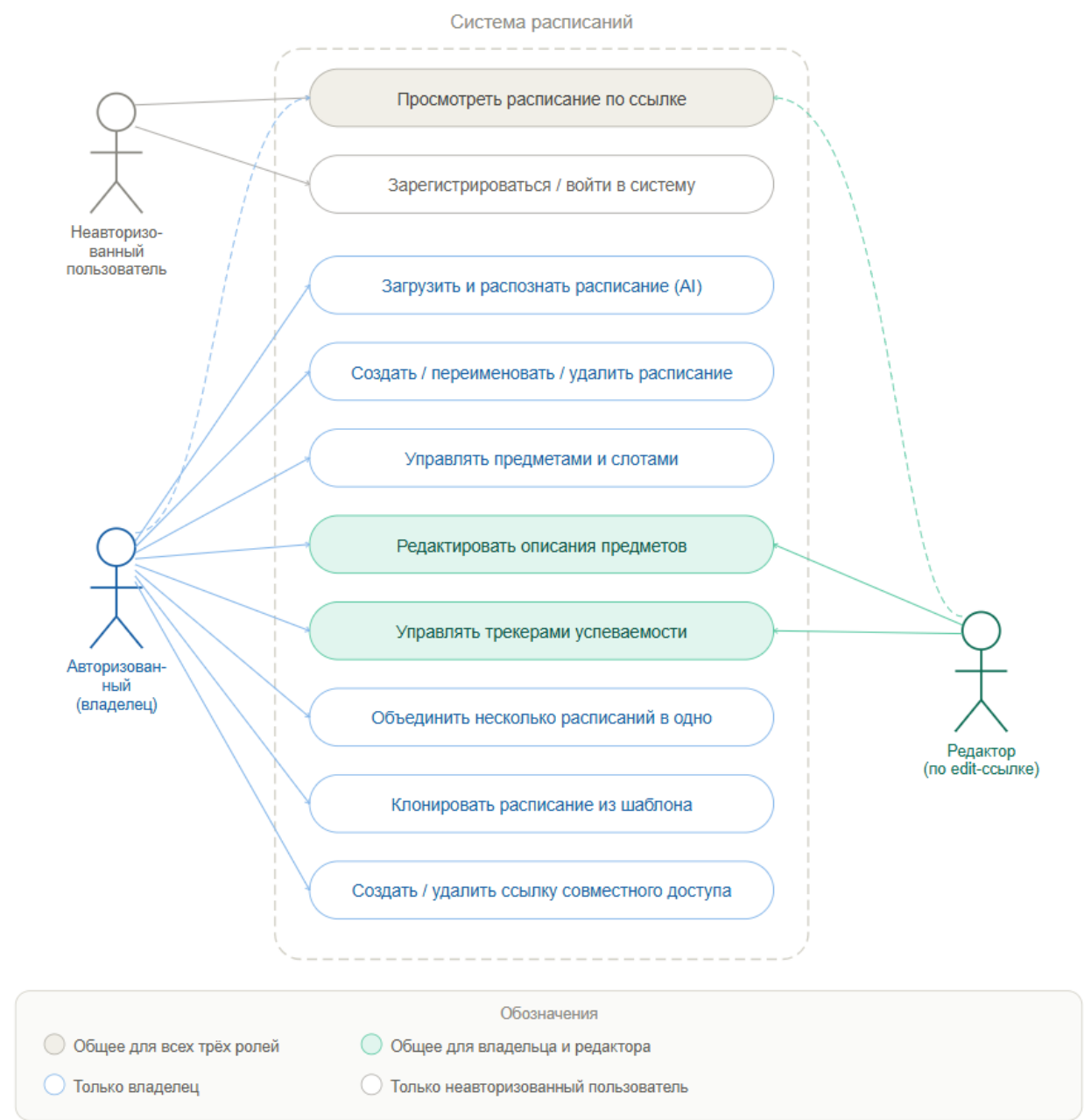


Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

База данных содержит шесть таблиц, образующих пять сущностей. Ниже приведено описание каждой из них.

Сущность `users` хранит учётные записи пользователей. Структура таблицы представлена в таблице 2.1.

Таблица 2.1 – Структура таблицы `users`

Поле	Тип	Описание
<code>id</code>	INTEGER	Первичный ключ, автоинкремент
<code>login</code>	VARCHAR(64)	Уникальный логин пользователя
<code>password_hash</code>	VARCHAR(256)	Хеш пароля (Werkzeug PBKDF2)
<code>created_at</code>	TIMESTAMP	Дата и время регистрации

Сущность `schedules` хранит расписания пользователей. Структура таблицы представлена в таблице 2.2.

Таблица 2.2 – Структура таблицы `schedules`

Поле	Тип	Описание
<code>id</code>	INTEGER	Первичный ключ
<code>user_id</code>	INTEGER	Внешний ключ на <code>users.id</code>
<code>name</code>	VARCHAR(128)	Название расписания
<code>data</code>	JSON	Структура расписания (предметы, слоты, типы)
<code>created_at</code>	TIMESTAMP	Дата создания

Сущность `subject_configs` хранит настройки предмета в контексте конкретного расписания: описание и флаг видимости. Структура таблицы представлена в таблице 2.3.

Таблица 2.3 – Структура таблицы `subject_configs`

Поле	Тип	Описание
<code>id</code>	INTEGER	Первичный ключ
<code>schedule_id</code>	INTEGER	Внешний ключ на <code>schedules.id</code>
<code>subject_name</code>	VARCHAR(256)	Название предмета
<code>description</code>	TEXT	Описание предмета (опционально)
<code>hidden</code>	BOOLEAN	Флаг скрытия предмета с дашборда

Сущность `metrics` хранит виджеты трекеров успеваемости. Структура таблицы представлена в таблице 2.4.

Таблица 2.4 – Структура таблицы `metrics`

Поле	Тип	Описание
<code>id</code>	INTEGER	Первичный ключ
<code>subject_config_id</code>	INTEGER	Внешний ключ на <code>subject_configs.id</code>
<code>type</code>	VARCHAR(32)	Тип трекера (одно из 11 значений)
<code>label</code>	VARCHAR(256)	Пользовательское название виджета
<code>config</code>	JSON	Конфигурация виджета (зависит от типа)
<code>progress</code>	JSON	Текущий прогресс виджета
<code>grid_col</code>	INTEGER	Столбец в CSS-сетке дашборда (0–11)
<code>grid_row</code>	INTEGER	Строка в CSS-сетке дашборда
<code>grid_w</code>	INTEGER	Ширина виджета в столбцах (1–12)
<code>grid_h</code>	INTEGER	Высота виджета в полустроках

Сущность `shares` хранит ссылки совместного доступа. **Сущность** `share_editors` связывает ссылку с пользователями, получившими права редактирования. Структура этих таблиц представлена в таблице 2.5.

Таблица 2.5 – Структура таблиц shares и share_editors

Таблица	Поле	Тип	Описание
shares	id	INTEGER	Первичный ключ
shares	schedule_id	INTEGER	Внешний ключ на schedules.id
shares	owner_id	INTEGER	Внешний ключ на users.id
shares	share_type	VARCHAR(16)	Тип доступа: view, edit, template
shares	token	VARCHAR(36)	UUID-токен для генерации ссылки
shares	created_at	TIMESTAMP	Дата создания ссылки
share_editors	share_id	INTEGER	Внешний ключ на shares.id
share_editors	user_id	INTEGER	Внешний ключ на users.id

На рисунке 2.2 представлена ER-диаграмма базы данных приложения.

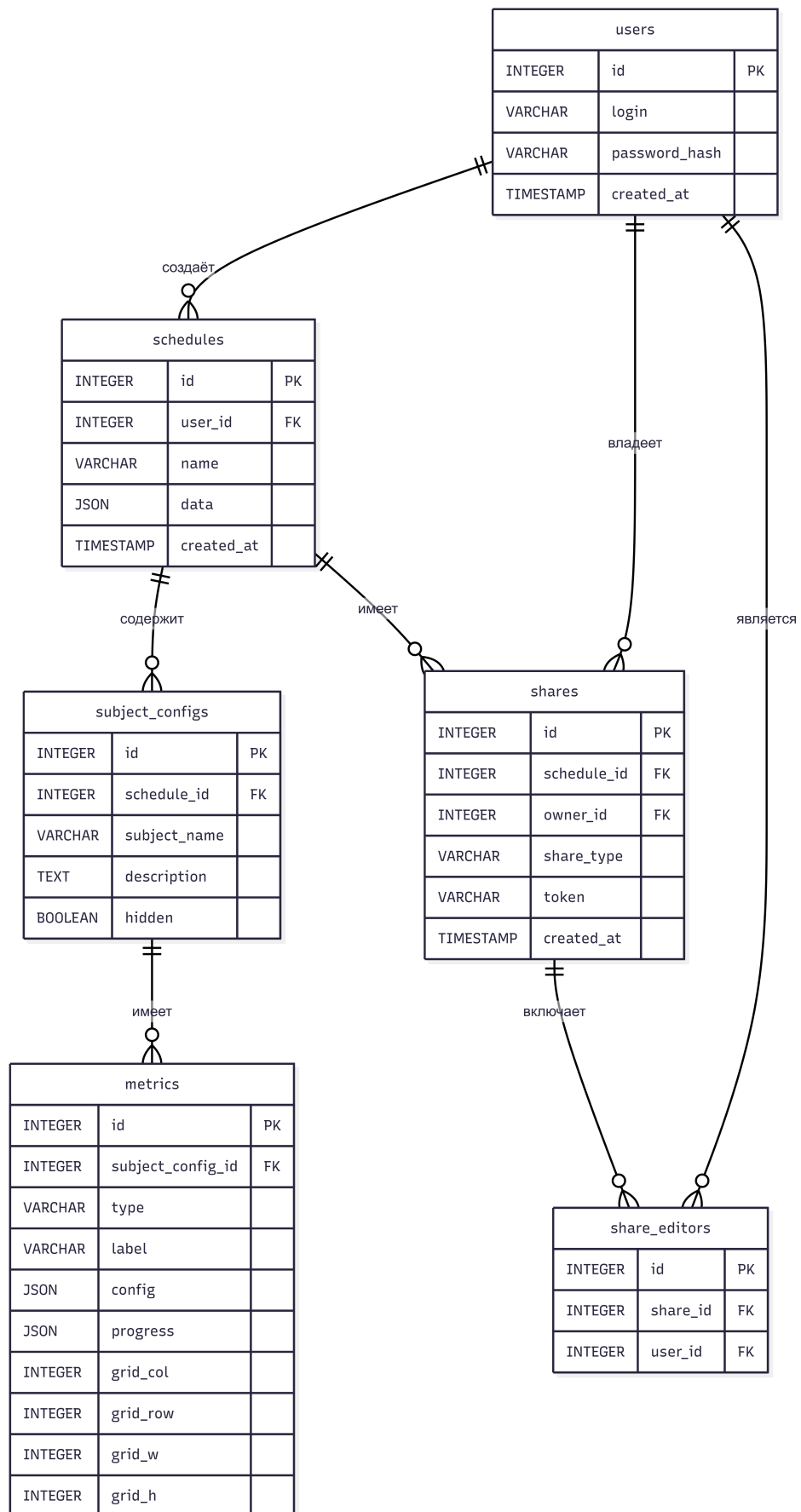


Рисунок 2.2 – ER-диаграмма базы данных приложения

2.4 Разработка макетов страниц

Интерфейс приложения включает следующие ключевые страницы.

Страница входа и регистрации. Минималистичная форма с полями логина и пароля, опцией «Запомнить меня» и ссылкой для перехода между формами.

Страница профиля. Список расписаний пользователя в виде карточек с названием и датой создания. Кнопки для открытия, удаления и перехода к загрузке нового расписания.

Страница загрузки расписания. Поле для перетаскивания изображения или выбора файла, поле дополнительных указаний для AI и кнопка запуска распознавания.

Страница просмотра расписания. Недельная таблица занятий с фильтрацией по неделям (периодам дат). Цветовая кодировка по типам занятий. Панель инструментов для редактирования слотов.

Страница предметов (дашборд метрик). Список предметов с раскрывающимися секциями. Внутри каждого предмета — сетка виджетов метрик. Кнопки для добавления трекера вручную или через описание на естественном языке (AI-генерация).

На рисунке 2.3 представлена страница входа, на рисунке 2.4 — страница регистрации, на рисунке 2.5 — страница профиля со списком расписаний, на рисунке 2.6 — страница загрузки расписания.

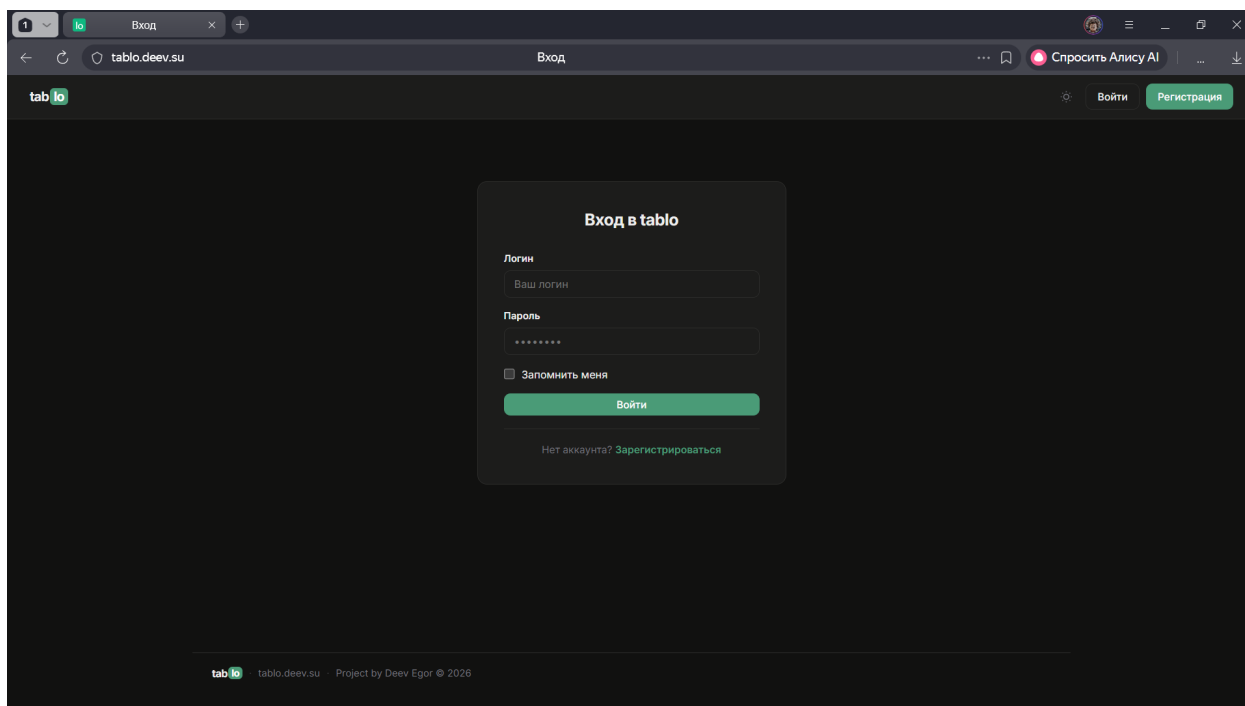


Рисунок 2.3 – Страница входа

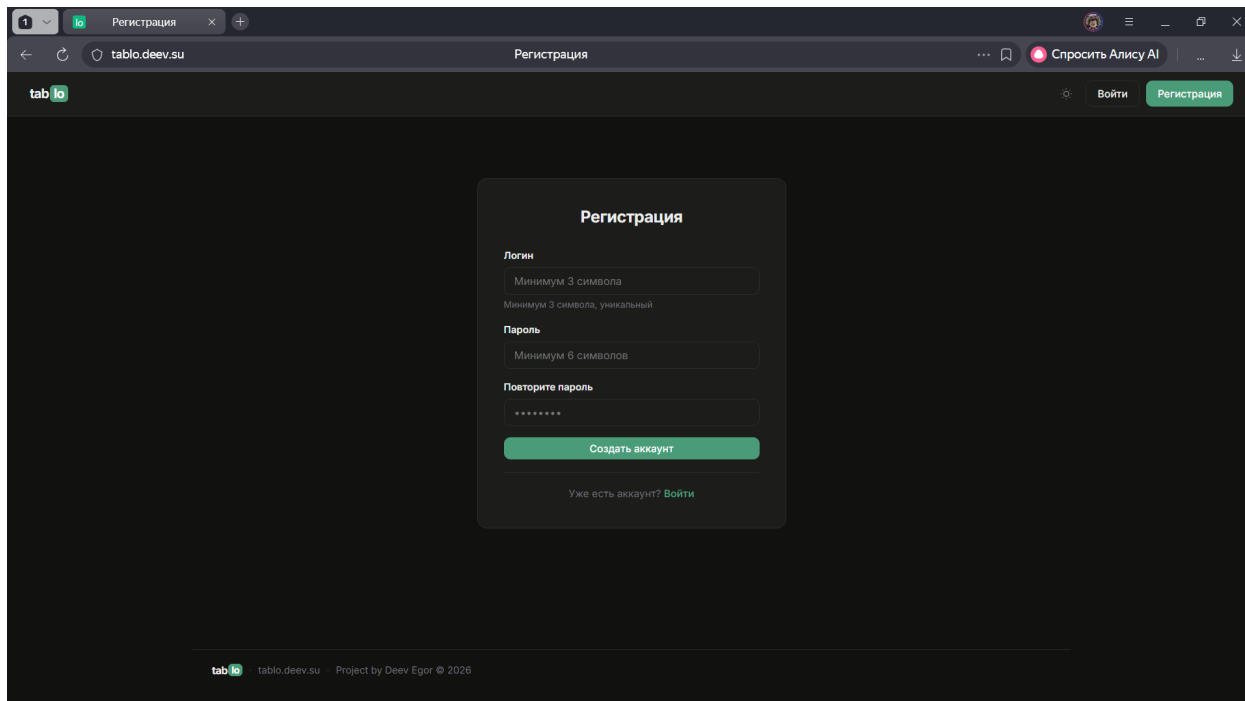


Рисунок 2.4 – Страница регистрации

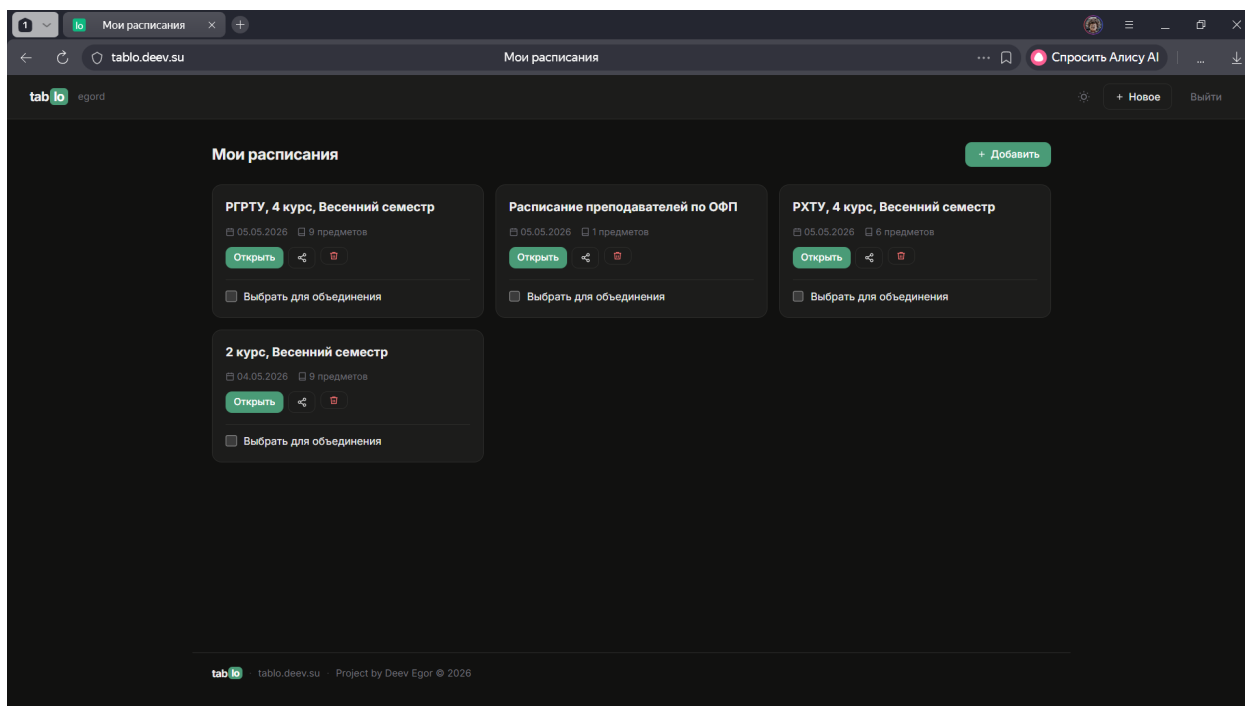


Рисунок 2.5 – Страница профиля со списком расписаний

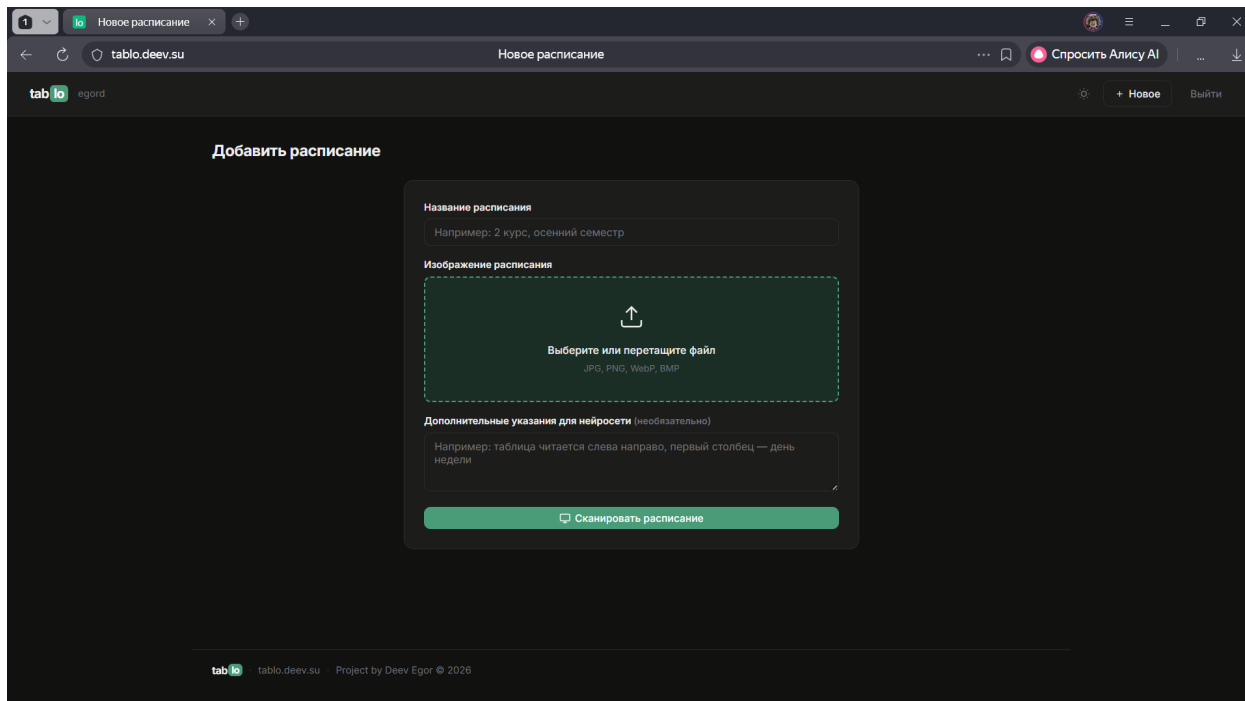


Рисунок 2.6 – Страница загрузки изображения расписания

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры проекта и вёрстка шаблонов

Проект организован по принципу пакетной структуры Flask: точка входа — файл `run.py`, который создаёт экземпляр приложения через фабричную функцию `create_app()`. Внутри пакета `app/` выделены четыре модуля: `models/` (описание сущностей), `routes/` (маршруты и обработчики), `services/` (бизнес-логика) и `templates/` (HTML-шаблоны Jinja2). Такое разделение обеспечивает независимость слоёв и упрощает сопровождение кода.

Фабричная функция инициализирует расширения Flask и регистрирует блюпринты, как показано в листинге 1.

Листинг 1 – Фабричная функция создания приложения (`app/__init__.py`)

```
1 def create_app():
2     app = Flask(__name__)
3     app.config.from_object('app.config.Config')
4
5     db.init_app(app)
6     migrate.init_app(app, db)
7     login_manager.init_app(app)
8     csrf.init_app(app)
9
10    from app.routes import auth, schedules, subjects, shares
11    app.register_blueprint(auth.bp)
12    app.register_blueprint(schedules.bp)
13    app.register_blueprint(subjects.bp)
14    app.register_blueprint(shares.bp)
15
16    return app
```

Шаблоны страниц строятся на базовом шаблоне `base.html` с использованием шаблонизатора Jinja2 [12], содержащем общую навигационную панель и подключение Bootstrap 5.3 [13]. Дочерние шаблоны переопределяют блоки `content` и `scripts`, что исключает дублирование разметки.

3.2 Реализация аутентификации и системы ролей

Аутентификация реализована с помощью библиотеки Flask-Login [8]. При успешном входе вызывается функция `login_user()`, которая сохраняет идентификатор пользователя в подписанной cookie-сессии. Пароли хранятся исключительно в виде хеша (алгоритм PBKDF2-SHA256 из библиотеки Werkzeug [7]): функция `set_password()` при регистрации и

check_password() при проверке. Функция регистрации представлена в листинге 2.

Листинг 2 – Обработчик регистрации (app/routes/auth.py)

```
1 @bp.route('/register', methods=['GET', 'POST'])
2 def register():
3     if request.method == 'POST':
4         login = request.form.get('login', '').strip()
5         password = request.form.get('password', '')
6         if User.query.filter_by(login=login).first():
7             flash('Логин уже занят', 'danger')
8         else:
9             user = User(login=login)
10            user.set_password(password)
11            db.session.add(user)
12            db.session.commit()
13            login_user(user)
14            return redirect(url_for('schedules.profile'))
15    return render_template('auth/register.html')
```

Ролевая модель реализована через проверку прав доступа перед каждой операцией с расписанием. Функции `_can_view()` и `_can_edit()` проверяют, является ли текущий пользователь владельцем расписания либо присутствует ли его идентификатор в таблице `share_editors` соответствующей ссылки совместного доступа.

3.3 Реализация CRUD-интерфейса расписаний и предметов

Все операции изменения структуры расписания реализованы как отдельные маршруты в блоке `schedules`. Данные расписания хранятся в поле `data` типа JSON в виде массива объектов-предметов. Каждый предмет содержит название, словарь типов занятий, и внутри каждого типа — словарь периодов дат со списком слотов. Формат одной записи слота: [день, время, кабинет, преподаватель].

Пример структуры данных одного предмета представлен в листинге 3.

Для операций с отдельным слотом используются разные HTTP-методы: добавление — PUT, изменение — PATCH, удаление — DELETE. Это позволяет Fetch API на клиентской стороне вызывать нужный обработчик без передачи явного поля «action».

Листинг 3 – Структура данных предмета в поле data

```
1 {  
2   "subject": "Математика",  
3   "types": {  
4     "Лекция": {  
5       "color": "blue",  
6       "dates": {  
7         "02.02-05.04": [  
8           "понедельник", "10:00-11:30", "Ауд. 123", "Иванов"  
9         ]  
10      }  
11    }  
12  }  
13 }
```

3.4 Сервис распознавания расписания с помощью AI

Сервис распознавания реализован в модуле `app/services/ai_scan.py`. При загрузке изображения выполняются следующие шаги:

1. Изображение считывается в байты и кодируется в base64.
2. Формируется запрос к AI Vision API: системный промпт задаёт строгую структуру ответа в формате JSON, пользовательское сообщение содержит изображение и дополнительные указания пользователя.
3. Ответ модели очищается от markdown-обёртки и разбирается функцией `json.loads()`.
4. Полученный массив объектов сохраняется в поле `data` новой записи таблицы `schedules`.

Ключевой фрагмент вызова API представлен в листинге 4.

Листинг 4 – Вызов AI Vision API для распознавания расписания (ai_scan.py)

```
1 response = client.chat.completions.create(  
2     model=model,  
3     messages=[  
4         {'role': 'system', 'content': SYSTEM_PROMPT},  
5         {  
6             'role': 'user',  
7             'content': [  
8                 {  
9                     'type': 'image_url',  
10                    'image_url': {  
11                        'url': f'data:{mime_type};base64,{  
image_data}',  
12                        'detail': 'high',  
13                    },  
14                },  
15                {'type': 'text', 'text': user_text},  
16            ],  
17        },  
18    ],  
19    temperature=0,  
20 )
```

Параметр `temperature=0` исключает случайность в ответах модели и обеспечивает воспроизводимый результат разбора. Использование OpenAI-совместимого интерфейса [10] позволяет сменить провайдера, изменив лишь переменные окружения `OPENAI_API_KEY` и `OPENAI_BASE_URL`.

3.5 Система трекеров успеваемости

Система метрик включает 11 типов виджетов, перечисленных в таблице 3.1.

Таблица 3.1 – Типы трекеров успеваемости

Тип	Назначение
checkpoints	Контрольные точки (лабы, тесты, задания)
progress_bar	Накопление баллов до максимального значения
counter	Простой счётчик с целью
stages	Последовательные этапы работы
checklist	Именованный список задач
attendance	Учёт посещаемости занятий
grades	Оценки с весами и автоматическим средним
deadlines	Задачи с датами дедлайнов
streak	Серия непрерывных дней активности
effort_hours	Учёт трудозатрат в часах
rating_history	История оценок в динамике

Каждый виджет хранит два JSON-поля: `config` (неизменяемые параметры, например, максимальное число баллов) и `progress` (текущее состояние, обновляемое пользователем). Виджеты отображаются на дашборде в виде CSS-сетки из 12 колонок; положение каждого виджета задаётся полями `grid_col`, `grid_row`, `grid_w` и `grid_h` в таблице `metrics`. Сохранение раскладки после перетаскивания осуществляется через запрос `Fetch API` к маршруту `/api/metrics/layout_batch`.

Добавление метрики через естественный язык реализовано в модуле `app/services/ai_metrics.py`: пользователь описывает желаемый трекер на русском языке, AI-модель [14] выбирает подходящий тип и генерирует конфигурацию, которая сразу сохраняется в базе данных.

3.6 Система совместного доступа и экспорта расписания

При создании ссылки совместного доступа в таблицу `shares` добавляется запись с уникальным `UUID`-токеном и выбранным типом доступа. Ссылка формируется по шаблону `/shared/{token}`. При переходе по ссылке система определяет тип доступа и применяет соответствующие права: для режима

template авторизованный пользователь может клонировать расписание себе в профиль.

На рисунке 3.1 представлена страница просмотра расписания с недельной таблицей занятий и блоком трекеров, на рисунке 3.2 — дашборд трекеров в режиме полноэкранного просмотра, на рисунке 3.3 — страница предметов с детальным видом слотов и трекером.

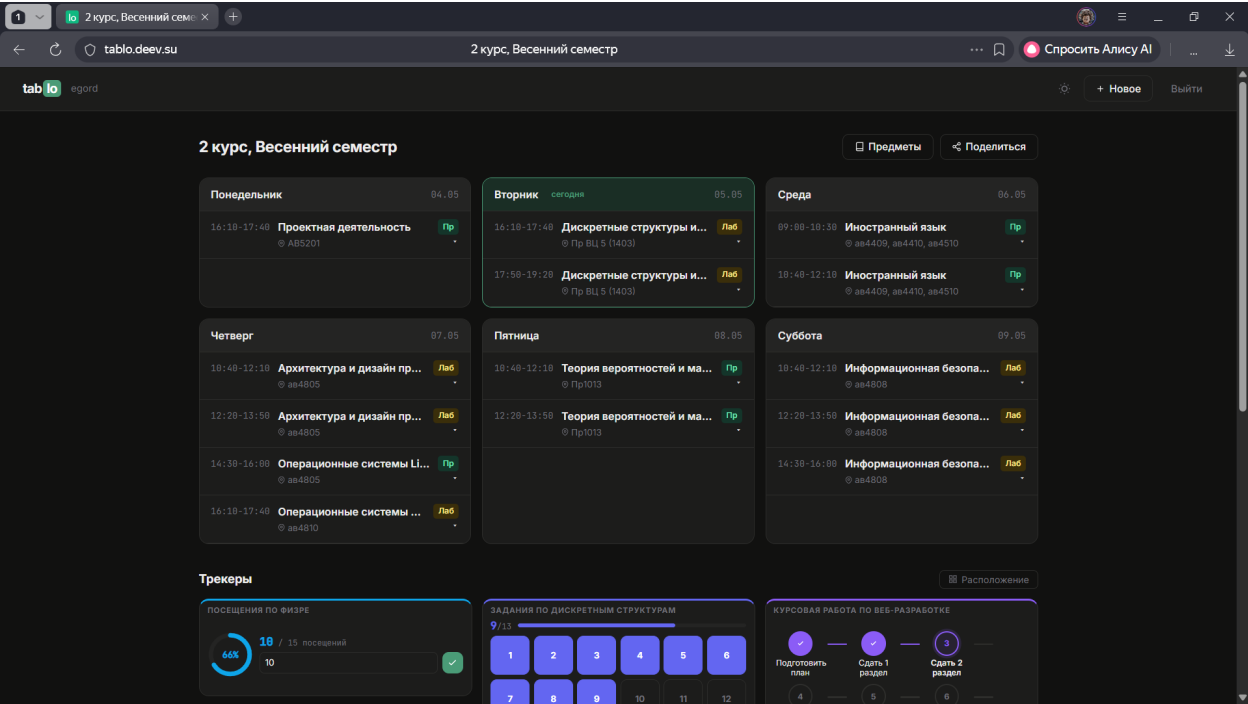


Рисунок 3.1 – Страница просмотра расписания

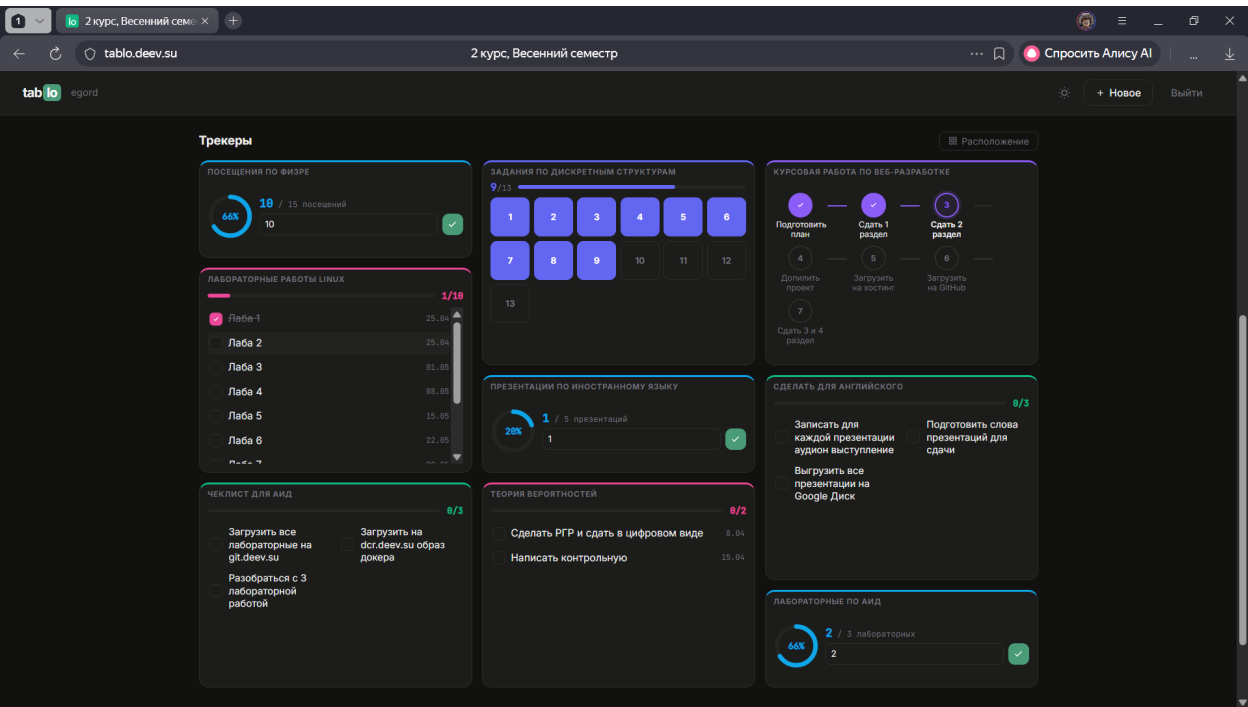


Рисунок 3.2 – Дашборд трекеров успеваемости

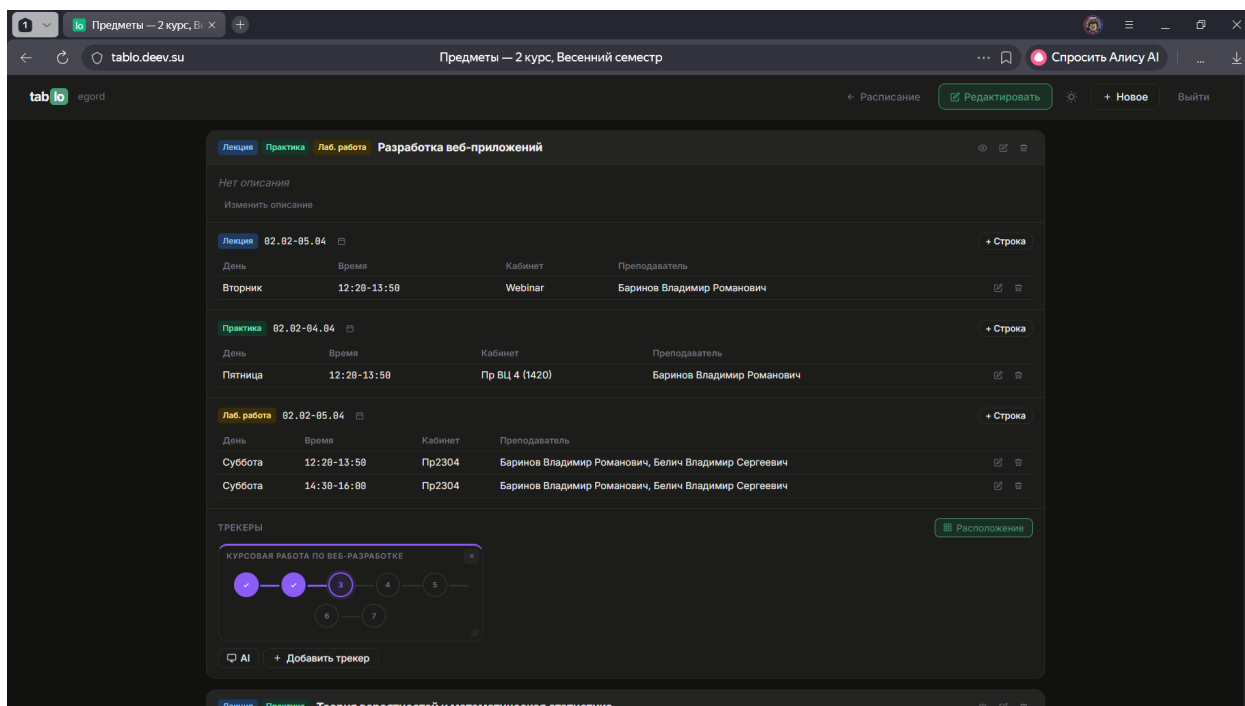


Рисунок 3.3 – Страница предметов с детальным видом слотов и трекером

Экспорт расписания доступен в четырёх форматах: JSON, CSV, PDF и PNG. Кнопка выгрузки расположена на карточке расписания на странице профиля в виде выпадающего меню. Скачать расписание может как его владелец, так и пользователь, перешедший по ссылке совместного доступа.

Форматы JSON и CSV формируются средствами стандартной библиотеки Python: для CSV вложенная структура разворачивается в плоскую таблицу с кодировкой UTF-8 с BOM для корректного отображения кириллицы в Microsoft Excel. Форматы PDF и PNG генерируются посредством серверного рендеринга HTML-шаблонов в headless-браузере Playwright [11]. PDF формируется из шаблона с табличной вёрсткой в альбомной ориентации A4. Для PNG используется отдельный шаблон, воспроизводящий недельную сетку карточек дней; активные периоды отбираются с учётом текущей даты — завершившиеся периоды исключаются из вывода.

На рисунке 3.4 представлен результат экспорта расписания в формате PNG.

2 курс, Весенний семестр					
Расписание · экспорт					
Понедельник		Среда		Четверг	
09:00-10:30 Основы технологического пред... @ Online курс		09:00-10:30 Иностранный язык @ av4409, av4410, av4510 А Индрикова Анастасия Александровна		10:40-12:10 Архитектура и дизайн програм... @ av4805 Недогарок Антон Александрович, Николаев Владислав Андреевич	
16:10-17:40 Проектная деятельность @ AB5201 А Владимир		10:40-12:10 Иностранный язык @ av4409, av4410, av4510 А Индрикова Анастасия Александровна		12:20-13:50 Архитектура и дизайн програм... @ av4805 Недогарок Антон Александрович, Николаев Владислав Андреевич	
				14:30-16:00 Операционные системы Linux @ av4805 Горшков Андрей Георгиевич, Самсонов Алексей Дмитриевич	
				16:10-17:40 Операционные системы Linux @ av4810 Горшков Андрей Георгиевич, Самсонов Алексей Дмитриевич	
Пятница		Суббота			
10:40-12:10 Теория вероятностей и математ... @ Пр1013 А Самохин Вячеслав Николаевич		10:40-12:10 Информационная безопасност... @ av4808 А Кривоногов Антон Алексеевич			
12:20-13:50 Теория вероятностей и математ... @ Пр1013 А Самохин Вячеслав Николаевич		12:20-13:50 Информационная безопасност... @ av4808 А Кривоногов Антон Алексеевич			
		14:30-16:00 Информационная безопасност... @ av4808 А Кривоногов Антон Алексеевич			

Рисунок 3.4 – Результат экспорта расписания в формате PNG

3.7 Публикация приложения

Исходный код проекта размещён в открытом репозитории на платформе GitHub по адресу <https://github.com/EDeev/tablo>. Репозиторий содержит полную структуру Flask-пакета, миграции базы данных, шаблоны и файл зависимостей `requirements.txt`.

Приложение развёрнуто на хостинге и доступно по адресу <https://tablo.deev.su>. Для обеспечения работоспособности на сервере настроены переменные окружения (`SECRET_KEY`, `DATABASE_URL`, `OPENAI_API_KEY`, `OPENAI_BASE_URL`), применены миграции базы данных через Flask-Migrate [15]. Для работы экспорта в форматах PDF и PNG установлен headless-браузер Chromium посредством команды `playwright install --with-deps chromium`.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы разработано веб-приложение для интерактивной работы с учебным расписанием на основе искусственного интеллекта. Все поставленные задачи выполнены в полном объёме.

На основе проведённых анализа и разработки можно сформулировать следующие **выводы**:

1. Анализ существующих систем показал, что ни одно из доступных решений не объединяет автоматическое распознавание расписания из изображения, персональные трекеры успеваемости и гибкую систему совместного доступа в рамках единого бесплатного инструмента. Это подтверждает актуальность и практическую значимость разработанного приложения.
2. Выбор технологического стека (Flask, PostgreSQL, Gemini 3.1 Flash через Timeweb Cloud AI) обоснован сравнительным анализом альтернатив по критериям гибкости, поддержки JSONB, доступности в российской инфраструктуре и стоимости запросов. Использование OpenAI-совместимого интерфейса обеспечивает переносимость кода при смене провайдера.
3. Архитектура базы данных из шести таблиц покрывает все требуемые сущности: пользователи, расписания, конфигурации предметов, метрики и ссылки совместного доступа. Хранение структуры расписания в поле типа JSON позволяет обрабатывать данные переменной вложенности без изменения схемы БД.
4. Система трекеров успеваемости из 11 типов виджетов обеспечивает гибкое агрегирование данных по предмету. Размещение виджетов на дашборде с CSS-сеткой 12 колонок и сохранением раскладки в базе данных обеспечивает персонализацию рабочего пространства каждого пользователя.
5. Ролевая модель «владелец — редактор — наблюдатель» реализована через механизм токен-ссылок и проверку прав доступа перед каждой операцией, что соответствует требованиям безопасности веб-приложений.

6. Реализован экспорт расписания в четырёх форматах: JSON (исходная структура данных), CSV (плоская таблица для табличных редакторов), PDF (печатный документ формата A4) и PNG (изображение недельной сетки занятий с фильтрацией по актуальным периодам). Форматы PDF и PNG формируются посредством headless-браузера Playwright.
7. Приложение опубликовано и доступно в сети интернет по адресу <https://tablo.deev.su>. Исходный код размещён в открытом репозитории <https://github.com/EDeev/tablo>, что обеспечивает воспроизводимость результатов и возможность дальнейшего развития проекта.

Рекомендации и предложения. В дальнейшем развитии приложения предлагается реализовать экспорт расписания в формат ICS для импорта в календарные приложения, добавить push-уведомления о предстоящих занятиях и дедлайнах, а также разработать мобильную версию интерфейса с поддержкой Progressive Web App.

Перспективы углубления темы. Накопленная база данных о типах занятий и трекерах успеваемости открывает возможность для реализации предиктивной аналитики: прогнозирования нагрузки по неделям и автоматических рекомендаций по расстановке приоритетов подготовки к экзаменам.

При разработке пояснительной записки к курсовому проекту соблюдались требования к структуре и правилам оформления [16].

СПИСОК ЛИТЕРАТУРЫ

- [1] Moodle HQ. About Moodle [Electronic resource]. — [S. l. : s. n.]. — URL: <https://moodle.com/about/> (online; accessed: 10.04.2026).
- [2] My Study Life Ltd. My Study Life — Student Planner App [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.mystudylife.com> (online; accessed: 10.04.2026).
- [3] Pallets Projects. Flask Documentation, version 3.1 [Electronic resource]. — [S. l. : s. n.]. — URL: <https://flask.palletsprojects.com/en/3.1.x/> (online; accessed: 12.04.2026).
- [4] PostgreSQL Global Development Group. PostgreSQL 16 Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/16/> (online; accessed: 12.04.2026).
- [5] SQLAlchemy Authors. SQLAlchemy 2.0 Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://docs.sqlalchemy.org/en/20/> (online; accessed: 12.04.2026).
- [6] Google LLC. Gemini API Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://ai.google.dev/gemini-api/docs> (online; accessed: 15.04.2026).
- [7] Pallets Projects. Werkzeug Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://werkzeug.palletsprojects.com/en/stable/> (online; accessed: 18.04.2026).
- [8] Max Countryman. Flask-Login Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://flask-login.readthedocs.io/en/latest/> (online; accessed: 18.04.2026).
- [9] WTForms Community. Flask-WTF Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://flask-wtf.readthedocs.io/en/stable/> (online; accessed: 18.04.2026).
- [10] OpenAI. OpenAI Python SDK Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://platform.openai.com/docs/libraries> (online; accessed: 20.04.2026).

- [11] Microsoft Corporation. Playwright for Python Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://playwright.dev/python/docs/intro> (online; accessed: 19.05.2026).
- [12] Pallets Projects. Jinja2 Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://jinja.palletsprojects.com/en/stable/> (online; accessed: 18.04.2026).
- [13] The Bootstrap Authors. Bootstrap 5.3 Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://getbootstrap.com/docs/5.3/> (online; accessed: 20.04.2026).
- [14] Timeweb Cloud. Timeweb Cloud AI — документация [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://timeweb.cloud/docs/ai> (дата обращения: 15.04.2026).
- [15] Miguel Grinberg. Flask-Migrate Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://flask-migrate.readthedocs.io/en/latest/> (online; accessed: 18.04.2026).
- [16] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.